Schleifen

Einführung in die Programmierung 1

Sommersemester 25

Überblick

- Schleifen in Java
- Verschachtelte Schleifen
- Programmierstil
- Programmiermuster
- Unterstützung durch die Entwicklungsumgebung

Schleifen in Java

Motivation

- Bisher fast nur Anweisungsfolgen (mit Verzweigungen)
- Viele Anweisungsfolgen müssen aber wiederholt ausgeführt werden
- Wiederholung muss im Programm formuliert werden

Schleifen

- Varianten in Java

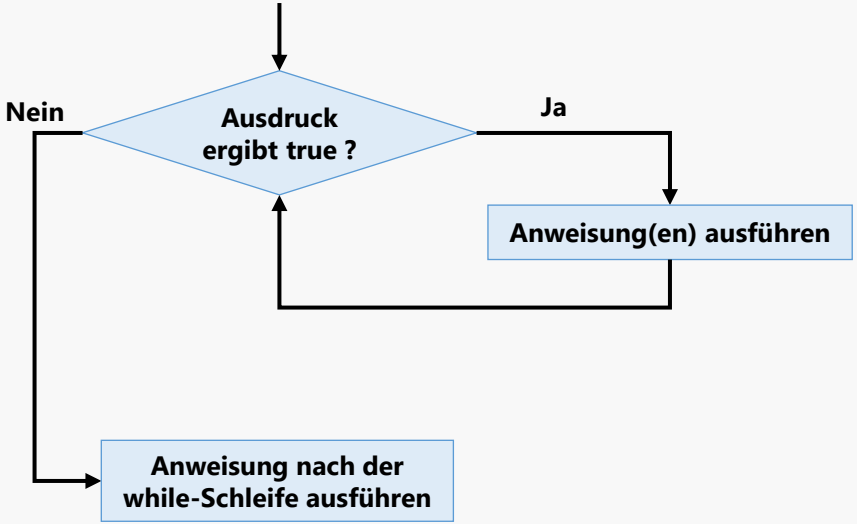
while	• Vor Schleifendurchlauf Ausdruck überprüfen • Dann Schleifenrumpf (Anweisung(en)) ausführen
do-while	• Zuerst Schleifenrumpf (Anweisung(en)) ausführen • Nach Schleifendurchlauf Ausdruck überprüfen
for	• Zuerst Vorbereitung (vor erstem Schleifendurchlauf) • Vor Schleifendurchlauf Ausdruck überprüfen • Dann Schleifenrumpf (Anweisung (en)) ausführen

while (1)

- Allgemeine Form

```
while (Ausdruck)
    Anweisung(en)
```

- Ausführung



while (2)

- Beliebiger Teil des Ausdrucks sollte in Anweisung(en) (Schleifenrumpf) verändert werden
  - Ansonsten kann der Wert des Ausdrucks im Schleifenkopf z. B. nicht von wahr auf falsch wechseln (Endlosschleife)
- Weitere Bezeichnungen
  - Abweisschleife
  - Vorprüfende oder kopfgesteuerte Schleife

Beispiel

```java
public class WhileTest {

    public static void main(String[] args) {
        int i = 0;
        System.out.println("Start:");
        while (i < 10) {
            System.out.println("i = " + i + " -> " + (i + 1) + ". Iteration");
            i++;
        }
        System.out.println("Ende!");
    }

}
```

```
Start:
i = 0 -> 1. Iteration
i = 1 -> 2. Iteration
i = 2 -> 3. Iteration
i = 3 -> 4. Iteration
i = 4 -> 5. Iteration
i = 5 -> 6. Iteration
i = 6 -> 7. Iteration
i = 7 -> 8. Iteration
i = 8 -> 9. Iteration
i = 9 -> 10. Iteration
Ende!
```

Beispiel (unbestimmte Anzahl an Schleifendurchläufen)

```java
int n = 17;
int count = 0;
while (n >= 1) {
    n /= 2;
    count++;
}
System.out.println(count);
```

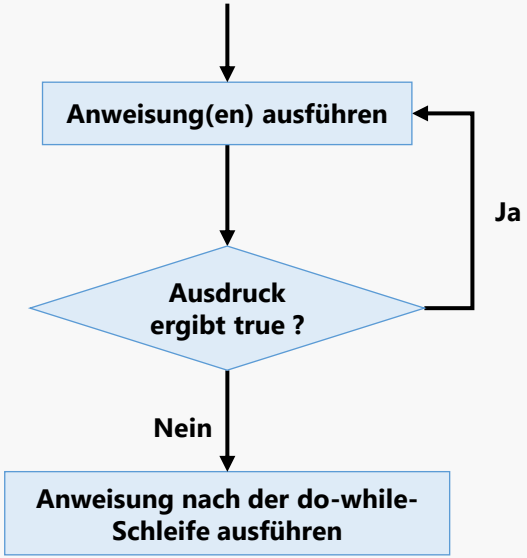
- Zählen, wie oft eine Zahl größer 0 durch 2 dividiert werden kann (Hinweis: zählt die Anzahl der Binärstellen einer Zahl größer 0)
- Input (in diesem Fall n) bestimmt die Anzahl der Schleifendurchläufe

do-while (1)

- Allgemeine Form

```
do
    Anweisung(en)
while (Ausdruck);
```

- Ausführung



do-while (2)

- Hinweis
  - Anweisung wird **mindestens einmal** ausgeführt
- Weitere Bezeichnungen
  - Durchlaufschleife
  - Nachprüfende oder fußgesteuerte Schleife

Beispiel (do-while)

```java
public class DoWhileTest {

    public static void main(String[] args) {
        int i = 0;
        System.out.println("Start:");
        do {
            System.out.println("i = " + i + " -> " + (i + 1) + ". Iteration");
            i++;
        } while (i < 10);
        System.out.println("Ende!");
    }

}
```

```
Start:
i = 0 -> 1. Iteration
i = 1 -> 2. Iteration
i = 2 -> 3. Iteration
i = 3 -> 4. Iteration
i = 4 -> 5. Iteration
i = 5 -> 6. Iteration
i = 6 -> 7. Iteration
i = 7 -> 8. Iteration
i = 8 -> 9. Iteration
i = 9 -> 10. Iteration
Ende!
```

Unterschied zwischen while und do-while

- Beispiel: Die Ziffern einer nicht negativen Zahl i in umgekehrter Reihenfolge ausgeben

Lösung mit while-Schleife	Lösung mit do-while-Schleife

…
`while (i > 0) {`
`  System.out.println(i % 10);`
`  i = i / 10;`
`}`
…
	…
	`do {`
	`  System.out.println(i % 10);`
	`  i = i / 10;`
	`} while (i > 0);`
	…

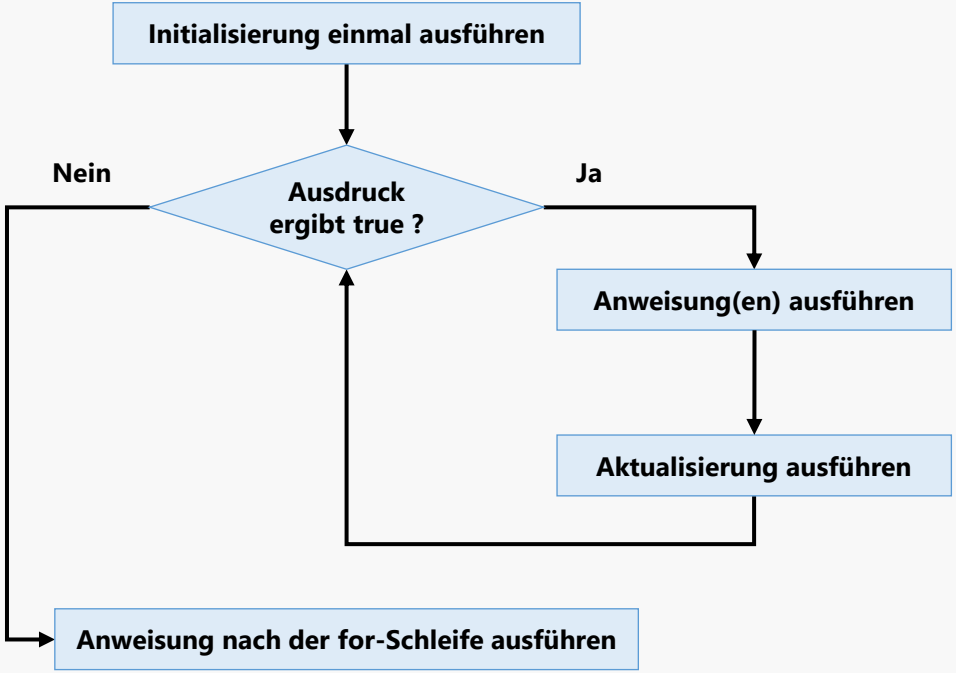
  - Lösung mit `do-while`-Schleife ist korrekt
  - Lösung mit `while`-Schleife hat ein Problem (0!)
    - Hinweis: i >= 0 löst hier nicht das Problem!

for (1)

- Allgemeine Form

```
for (Initialisierung; Ausdruck; Aktualisierung)
    Anweisung(en)
```

- Ausführung



for (2)

- Jeder der drei Teile im Schleifenkopf kann weggelassen werden
  - https://docs.oracle.com/javase/specs/jls/se23/html/jls-14.html#jls-14.14.1
- Weitere Bezeichnung
  - Zählschleife (Sonderform der vorprüfenden Schleife)

Beispiel (for)

```java
public class ForTest {

    public static void main(String[] args) {
        System.out.println("Start:");
        for (int i = 0; i < 10; i++) {
            System.out.println("i = " + i + " -> " + (i + 1) + ". Iteration");
        }
        System.out.println("Ende!");
    }

}
```

```
Start:
i = 0 -> 1. Iteration
i = 1 -> 2. Iteration
i = 2 -> 3. Iteration
i = 3 -> 4. Iteration
i = 4 -> 5. Iteration
i = 5 -> 6. Iteration
i = 6 -> 7. Iteration
i = 7 -> 8. Iteration
i = 8 -> 9. Iteration
i = 9 -> 10. Iteration
Ende!
```

Sichtbarkeit von Variablen

- Sichtbarkeitsbereich einer Variable
  - Programmstück, in dem auf die Variable zugegriffen werden kann
- Variable in einem Block (z. B. Block einer `for`-Schleife)
  - Sichtbar von der Deklaration bis zum Ende des Blocks, in dem sie deklariert wurde, z. B.

```
for (int i = 0; i < 100; i++) {
    …
    int x = 20;
    …
}
…
System.out.println(i + x);
```

Ab hier ist i in der Schleife bekannt

Ab hier ist x im aktuellen Schleifendurchlauf bekannt

Hier sind i und x **nicht mehr** bekannt

Gibt einen Übersetzungsfehler!

Beispiele (Zählschleifen)

- Aufsteigend

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

Zahlen von 0 bis 9 auf der Konsole ausgeben

- Absteigend

```
for (int i = 10; i >= 0; i--) {
    System.out.println(i);
}
```

Zahlen von 10 bis 0 auf der Konsole ausgeben

- Nicht triviales Inkrement

```
for (long i = 1; i < 1000000000000L; i *= 3) {
    System.out.println(i);
}
```

Alle Potenzen von 3, die kleiner als 10^{12} sind, ausgeben

Ein Beispiel – unterschiedliche for-Schleifen

- 10! berechnen

`int factorial = 1;`
`for (int i = 2; i <= 10; i++) {`
`  factorial = factorial * i;`
`}`
	• Einfache, lesbare Variante
	• Typische for-Schleife

`int factorial, i;`
`for (factorial = 1, i = 2; i <= 10; factorial = factorial * i, i++)`
`  ;`
	• Mehrere, durch Beistrich getrennte, Ausdrücke in der Initialisierung und Aktualisierung
	• Kein Schleifenrumpf notwendig
	• Schlechter lesbar

`for (int factorial = 1, i = 2; i <= 10; factorial = factorial * i++);`	• Möglich, aber **sinnlos** • Variable `factorial` ist außerhalb der Schleife nicht verwendbar

`int factorial = 1, i = 2;`
`for (; i <= 10;) {`
`  factorial = factorial * i++;`
`}`
	• Initialisierung und Aktualisierung bleiben leer
	• Initialisieren bei der Deklaration
	• Aktualisieren im Schleifenrumpf

Beispiel – Konvertierung in Binärzahl (1)

- Es sei eine ganze positive Zahl n gegeben
- Konvertiere n in die entsprechende Binärdarstellung
- Beispiel
  - n = 105 -> Ergebnis: 1101001
- Ablauf
  - Immer wieder durch 2 dividieren (bis Ergebnis = 0)
    - Rest (0 oder 1) berücksichtigen
  - Ergebnis der Division im nächsten Schritt benutzen
  - Aber: Ausgabe im Ablauf ergibt verkehrtes Muster

Operation	Rest
105 : 2 = 52	1
52 : 2 = 26	0
26 : 2 = 13	0
13 : 2 = 6	1
6 : 2 = 3	0
3 : 2 = 1	1
1 : 2 = 0	1

Beispiel – Konvertierung in Binärzahl (2)

- Einfache Lösung mit verkehrter Ausgabe

```java
public class BinarySimple {

    public static void main(String[] args) {
        int n = 105;
        while (n > 0) {
            if (n % 2 == 0) {
                System.out.print("0");
            } else {
                System.out.print("1");
            }
            n /= 2;
        }
    }

}
```

```
1001011
```

Beispiel – Konvertierung in Binärzahl (3)

- Alternative Lösung mit korrekter Ausgabe

```java
public class Binary {
    public static void main(String[] args) {
        int n = 105;
        int power = 1;
        while (power <= n / 2) {
            power *= 2;
        }
        while (power > 0) {
            if (n < power) {
                System.out.print(0);
            } else {
                System.out.print(1);
                n -= power;
            }
            power /= 2;
        }
    }
}
```

power auf die größte 2er-Potenz <= n setzen

1101001

Überprüfen, ob eine bestimmte 2er-Potenz (von groß zu klein) in n vorhanden ist. Wenn vorhanden, dann wird 1 ausgegeben und diese 2er-Potenz subtrahiert, ansonsten wird 0 ausgegeben.

break und continue

- break
  - Führt zum sofortigen Ausstieg aus dem aktuellen Schleifendurchlauf und damit zum Ausstieg aus der Schleife
- continue
  - Überspringt die restlichen Anweisungen im aktuellen Schleifendurchlauf und geht wieder zum Ausdruck

break und continue – Vergleich (for-Schleife)

```java
int n = 10;
for (int i = 1; i < n; i++) {
    if (i % 5 == 0) {
        break;
    }
    System.out.println(i);
}
```

Wenn i den Wert 5 hat, wird **die Schleife** abgebrochen

```
1
2
3
4
```

```java
int n = 10;
for (int i = 1; i < n; i++) {
    if (i % 5 == 0) {
        continue;
    }
    System.out.println(i);
}
```

Wenn i den Wert 5 hat, wird **der Schleifendurchlauf** abgebrochen. Wichtig: Das Update i++ wird aber ausgeführt!

```
1
2
3
4
6
7
8
9
```

break und continue – Vergleich (while-Schleife)

```java
int n = 10;
int i = 1;
while (i < n) {
    if (i % 5 == 0) {
        break;
    }
    System.out.println(i);
    i++;
}
```

Wenn i den Wert 5 hat, wird **die Schleife** abgebrochen

```
1
2
3
4
```

```java
int n = 10;
int i = 1;
while (i < n) {
    if (i % 5 == 0) {
        i++;
        continue;
    }
    System.out.println(i);
    i++;
}
```

Wenn i den Wert 5 hat, wird **der Schleifendurchlauf** abgebrochen. Wichtig: Das Update i++ muss explizit davor ausgeführt werden!

```
1
2
3
4
6
7
8
9
```

Beispiel (Primzahltest mit vorzeitigem Abbruch)

```java
public class PrimeTest {

    public static void main(String[] args) {
        long n = 97; // Simulate input data
        boolean isPrime = n >= 2;
        for (long factor = 2; factor < n; factor++) {
            if (n % factor == 0) {
                isPrime = false;
                break;
            }
        }
        if (isPrime) {
            System.out.println(n + " is prime");
        } else {
            System.out.println(n + " is not prime");
        }
    }

}
```

An dieser Stelle ist das Ergebnis klar und die Schleife muss nicht weiter ausgeführt werden

Verschachtelte Schleifen

Verschachtelte Schleifen

- Eine Schleife ist eine Anweisung
- Daher kann eine Schleife auch in einer Schleife vorkommen

Beispiel (2D-Muster auf der Konsole ausgeben)

```java
int n = 3;
int m = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*****
*****
*****
```

Entwicklung der Zählvariablen

```
i:0 j:0
i:0 j:1
i:0 j:2
i:0 j:3
i:0 j:4

i:1 j:0
i:1 j:1
i:1 j:2
i:1 j:3
i:1 j:4

i:2 j:0
i:2 j:1
i:2 j:2
i:2 j:3
i:2 j:4
```

Weitere Beispiele für Ausgaben

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*****
****
***
**
*
```

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print("*");
    }
    for (int j = i; j < n; j++) {
        System.out.print("#");
    }
    System.out.println();
}
```

```
#####
*####
**###
***##
****#
```

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*
**
***
****
*****
```

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(i + j);
    }
    System.out.println();
}
```

```
01234
12345
23456
34567
45678
```

Beispiel (Multiplikationstabelle, Angabe)

- Eine Multiplikationstabelle für das kleine Einmaleins in der folgenden Form ausgeben

*	1	2	…	9	10
1	1	2	…	9	10
2	2	4	…	18	20
3	3	6	…	27	30
4	4	8	…	36	40
5	5	10	…	45	50
6	6	12	…	54	60
7	7	14	…	63	70
8	8	16	…	72	80
9	9	18	…	81	90
10	10	20	…	90	100

Beispiel (Multiplikationstabelle, Lösung)

```java
public class Multiplication {

    public static void main(String[] args) {
        int n = 10;
        System.out.print("*\t");
        for (int i = 1; i <= n; i++) {
            System.out.print("\t" + i);
        }
        System.out.println("\n");
        for (int i = 1; i <= n; i++) {
            System.out.print(i + "\t");
            for (int j = 1; j <= n; j++) {
                System.out.print("\t" + i * j);
            }
            System.out.println();
        }
    }

}
```

Beispiel für komplexeres Muster (1)

- Folgendes Muster sollte erzeugt werden (für gegebenes n)

n = 2

```
v-v
-v-
```

n = 3

```
v---v
-v-v-
--v--
```

n = 4

```
v-----v
-v---v-
--v-v--
---v---
```

- Analyse
  - Es werden n Zeilen ausgegeben
  - Jede Zeile enthält 2*n – 1 Zeichen (Anzahl der Spalten bzw. Breite)
  - Beispiel: Position der „v“-Zeichen für n= 4

Zeile	Spalte links	Spalte rechts
1	1 (= Zeile)	7 (= Breite - 0 = Breite - Spalte links + 1)
2	2 (= Zeile)	6 (= Breite - 1 = Breite - Spalte links + 1)
3	3 (= Zeile)	5 (= Breite - 2 = Breite - Spalte links + 1)
4	4 (= Zeile)	4 (= Breite - 3 = Breite - Spalte links + 1)

Beispiel für komplexeres Muster (2)

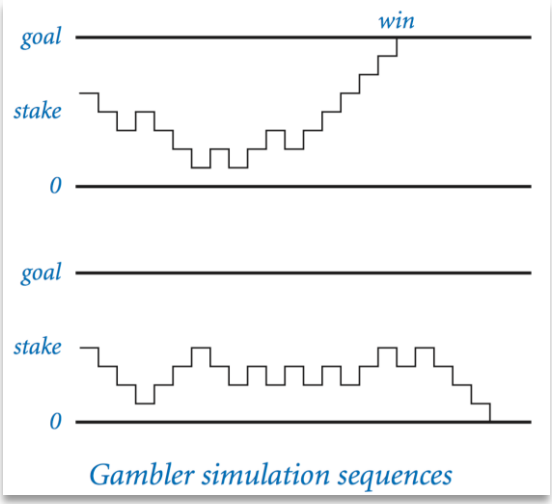
```java
public class VPattern {
    public static void main(String[] args) {
        int n = 4;
        int width = 2 * n - 1;
        for (int row = 1; row <= n; row++) {
            for (int column = 1; column <= width; column++) {
                if (column == row || width - column + 1 == row) {
                    System.out.print('v');
                } else {
                    System.out.print('-');
                }
            }
            System.out.println();
        }
    }
}
```

Beispiel (Simulation)

- Spiel beginnt mit einem Startbetrag (stake)
- In jeder Runde wird 1 Euro eingesetzt
  - 50 % Chance auf Gewinn von einem Euro (Einsatz zurück + 1 Euro)
  - 50 % Chance auf Verlust des Einsatzes (- 1 Euro)
- Es wird am Anfang ein Zielbetrag (goal) festgelegt
  - Wird dieser erreicht, dann wird das Spiel beendet

Beispiel (Simulation)

- Simulation
  - Zufälliger Entscheid, ob Gewinn oder Verlust
  - Abbruch, wenn Zielbetrag erreicht oder kein Geld mehr vorhanden
  - Mehrmals wiederholen (trials) und Mittelwerte berechnen
- Code in TUWEL
  - `GamblingSimulation.java`



https://introcs.cs.princeton.edu/java/13flow/images/gambler.png

Programmierstil

Schleifenrumpf

- Schleifenrumpf immer als Block implementieren
  - Klammern mit Einrückungen erhöhen Lesbarkeit
  - Schleifenrumpf kann leicht erweitert werden

break und continue

- In Schleifen **nur sparsam** verwenden
- Können übermäßige Verschachtelungen vermeiden
- Können oft durch eine alternative Formulierung des Ausdrucks umgangen werden

Beispiel für break und seine Vermeidung (1)

- Mit break

```
int n = 1000;
int factor = 2;
while (factor <= n) {
    if (n % factor == 0) {
        break;
    }
    factor++;
}
System.out.println("The smallest factor other than 1 for " + n + " is " + factor);
```

Abbruch, wenn ein Faktor != 1 gefunden wurde.
Mit break kürzer als untere Version.

- Ohne break

```
int n = 1000;
int factor = 2;
boolean found = false;
while (factor <= n && !found) {
    if (n % factor == 0) {
        found = true;
    } else {
        factor++;
    }
}
System.out.println("The smallest factor other than 1 for " + n + " is " + factor);
```

Version ohne break. Benötigt eine zusätzliche Variable und ist textuell länger. Dafür kann der Ausstieg aus der Schleife nur an einer Stelle geschehen (Überprüfung der Schleifenbedingung).

Beispiel für break und seine Vermeidung (2)

- Anderer Schleifenansatz (kein break erforderlich, leichter lesbar)

```
int n = 1000;
int factor = 2;
while (n % factor != 0) {
    factor++;
}
System.out.println("The smallest factor other than 1 for " + n + " is " + factor);
```

- Nicht immer so einfach möglich!

Programmiermuster

Auswahl einer Schleife (1)

- `for`-Schleife
  - Vorgegebener Start- und Endwert
  - Fixe Schrittweite
- `while`-Schleife
  - Lediglich Endkriterium gegeben
  - Nicht vorhersagbar, wie oft die Schleifenanweisungen ausgeführt werden müssen, bis das Endkriterium zutrifft

Auswahl einer Schleife (2)

- Transformation
  - Die verschiedenen Schleifenarten lassen sich auch ineinander transformieren
- Anwendung
  - Abhängig vom Problem ist meist eine der Schleifen leichter ausformulierbar (lesbar)

Beispiele für Transformationen (beide Richtungen!)

while-Schleife	entsprechende do-while-Schleife

`while (ausdr)`
`  Schleifenanweisung;`
	`if (ausdr)`
	`  do`
	`  Schleifenanweisung;`
	`  while (ausdr);`

do-while-Schleife	entsprechende while-Schleife

`do`
`  Schleifenanweisung;`
`while (ausdr);`
	`Schleifenanweisung;`
	`while (ausdr)`
	`  Schleifenanweisung;`

for-Schleife	entsprechende while-Schleife

`for (ausdr1; ausdr2; ausdr3)`
`  Schleifenanweisung;`
	`ausdr1;`
	`while (ausdr2) {`
	`  Schleifenanweisung;`
	`  ausdr3;`
	`}`

Akkumulierende Schleifen (1)

- Schleifen, mit denen ein Ergebnis konstruiert wird
- Schema
  - Akkumulierende Variable vor der Schleife initialisieren
  - Akkumulierende Variable in der Schleife verändern
  - Nach der Schleife Ergebnis weiterverwenden
- Beispiele

```java
int sum = 0;
int i = 1;
while (i <= 10) {
    sum += i;
    i++;
}
System.out.println(sum);
```

```java
int sum = 0;
for (int i = 1; i <= 10; i++) {
    sum += i;
}
System.out.println(sum);
```

Akkumulierende Schleifen (2)

- Die Summe aller geraden Zahlen im Intervall [2, 100] berechnen und ausgeben

```java
public class SpecialSum {

    public static void main(String[] args) {

        int sum = 0;
        int counter = 2;

        while (counter <= 100) {
            if ((counter % 2) == 0) {
                sum = sum + counter;
            }
            counter = counter + 1;
        }

        System.out.println("Sum: " + sum);
    }

}
```

```java
public class Specialsum {

    public static void main(String[] args) {
        int sum = 0;
        for (int counter = 2; counter <= 100; counter += 2) {
                sum = sum + counter;
        }
        System.out.println("Sum: " + sum);
    }

}
```

Kürzere Variante mit for-Schleife – startet mit einer geraden Zahl und zählt immer 2 beim Update dazu

Zaunpfahlproblem – Motivation

- Beispiel
  - Die Zahlen von 1 bis n durch ein Komma getrennt ausgeben
- Falsche Lösung

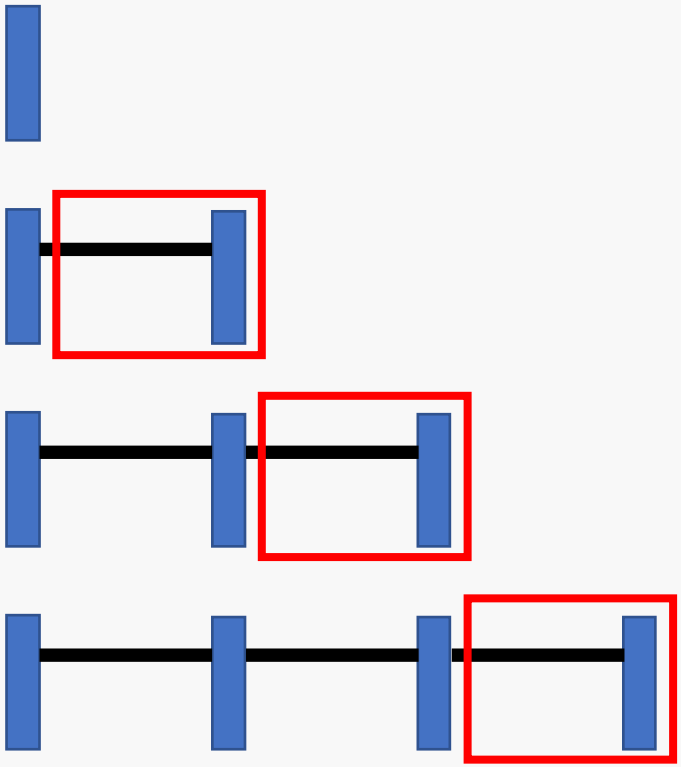
```java
int n = 5
for (int i = 1; i <= n; i++) {
    System.out.print(", " + i);
}
System.out.println();
```

```
, 1, 2, 3, 4, 5
```

- Problem
  - n Zahlen werden ausgegeben, aber nur n-1 Kommas werden benötigt
  - `System.out.print(i + ", ");` löst nicht das Problem!
    - Ergibt `1, 2, 3, 4, 5,`

Zaunpfahlproblem – Analogie

- Analogie Zaun
  - 4 Pfähle, 3 Streben
- Aufbau
  - Einen Pfahl fixieren
  - Wiederhole
    - Strebe + nächsten Pfahl hinzugeben



Zaunpfahlproblem – Lösungen

- 1. Variante
  - Vor der Schleife Start fixieren
    - Siehe vorherige Folie

```java
int n = 5;
System.out.print(1);
for (int i = 2; i <= n; i++) {
    System.out.print(", " + i);
}
System.out.println();
```

- 2. Variante
  - Nach der Schleife abschließen

```java
int n = 5;
for (int i = 1; i <= n - 1; i++) {
    System.out.print(i + ", ");
}
System.out.println(n);
```

- 3. Variante
  - Mit Bedingung Ausgabe steuern

```java
int n = 5;
for (int i = 1; i <= n; i++) {
    System.out.print(i);
    if (i < n) {
        System.out.print(", ");
    }
}
System.out.println();
```

```
1, 2, 3, 4, 5
```

Unterstützung durch die Entwicklungsumgebung

for-Schleifen automatisch generieren

- fori Live-Template

```
fori
fori                                  Create iteration loop
Press Strg+Leertaste to see non-imported classes  Next Tip
```

```
for (int i = 0; i < ; i++) {

}
```

```
for (int counter = 0; counter < 100; counter++) {

}
```

Warnungen (kleine Auswahl an Beispielen)

- Zu viele Iterationen

```
for (int i = 0; i < 10; i--) {
    Loop executes zero or billions of times
}
```

- Update hat keine Auswirkung (Endlosschleife)

```
for (int i = 0; i < 10; i = i * 2) {
    sum += i;
}
```

Result of 'i * 2' is always '0'

Replace with '0' Alt+Umschalt+Eingabe More actions... Alt+Eingabe

- Kein Update der Zählvariable (Endlosschleife)

```
int i = 0;
while (i < 10) {
    sum +=
}
```

Condition 'i < 10' is always 'true'

Simplify 'i < 10' to true Alt+Umschalt+Eingabe More actions... Alt+Eingabe

Was passiert hier?

```java
public class StarPattern {
    public static void main(String[] args) {
        int n = 2;
        for (int i = -n; i <= n; i++) {
            for (int j = -n; j < n; j++) {
                if (i * i < j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }
}
```

Ausgabe, die wir erwarten	Ausgabe, die wir bekommen

`*  *`
`* *  * *`
`* * * * *`
`* *  * *`
`*  *`
	
	`*`
	`* *  *`
	`*`

Debugging

- Debugger
  - Werkzeug zum Diagnostizieren und Auffinden von Fehlern in Programmen
- Typische Funktionen
  - Steuerung des Programmablaufs (Haltepunkte, engl. Breakpoints)
  - Schrittweise Durchführung von Programmen
  - Inspizieren und modifizieren von Variablen

Debugger

```
public class StarPattern {

    public static void main(String[] args) {   args: []
        int n = 5;   n: 5
        for (int i = -n; i <= n; i++) {   i: -5   i: -5
            for (int j = -n; j <= n; j++) {   n: 5
                if (i * i <= j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
```

Debugger einschalten

Breakpoint

Einzelne Programmschritte ausführen

Variablen inspizieren

Werte von Ausdrücken inspizieren

- Mauszeiger auf gewünschten Ausdruck (Operator) bewegen

```
8
9   int result = a + b * c;   a: 2   b: 3   c: 5
10
11  }
```

15

Maus über *

```
8
9   int result = a + b * c;   a: 2   b: 3   c: 5
10
11  }
```

17

Maus über +

Live-Beispiel

- Live-Beispiel für einfachen Debugger-Durchlauf
  - Breakpoints an verschiedenen Stellen setzen
  - Schrittweiser Ablauf
  - Variablen inspizieren

```java
public class StarPattern {

    public static void main(String[] args) {
        int n = 5;
        for (int i = -n; i <= n; i++) {
            for (int j = -n; j < n; j++) {
                if (i * i < j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }

}
```

Was passiert hier ? – Lösung

```java
public class StarPattern {

    public static void main(String[] args) {
        int n = 2;
        for (int i = -n; i <= n; i++) {
            for (int j = -n; j <= n; j++) {
                if (i * i <= j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }

}
```

```
*       *
* *   * *
* * * * *
* *   * *
*       *
```

Zusammenfassung

- Schleifen in Java
- Verschachtelte Schleifen
- Programmierstil
- Programmiermuster
- Unterstützung durch die Entwicklungsumgebung